

Answers

1- Depth-First Search:

Is a graph search method that explores as far as possible in one direction before going back and trying another path.

Breadth-First Search:

Is a graph search method that explores all close vertices before moving farther.

Graph:

Is a data structure made of vertices and edges. The vertices represent points or objects, and the edges represent connections between them.

Adjacent Vertices:

Vertices that are connected directly to each other by an edge, usually the cells directly above, below, left, or right of a cell.

2- In my own words, the problem statement is asking to determine whether a program can find a valid path from one point in a two-dimensional grid to another point.

The program must search through the grid while avoiding blocked locations and decide if the ending point can be reached from the starting point.

3- Forecasting

a) Depth-first search explores one path as far as it can before backtracking and trying another path. Breadth-first search explores all nearby points first before moving farther away from the starting point. DFS goes deep first, while BFS goes level by level.

b) Depth-first search usually uses a stack or recursion. Breadth-first search usually uses a queue. Both algorithms also need a way to keep track of visited points so the same location is not searched more than once.

c)

1. Start at the starting point.
2. Push it onto a stack.
3. Remove the top point from the stack while the stack is not empty
4. The search is successful if the point is the ending point
5. Mark it as visited if the point has not been visited
6. Check each valid adjacent point.
7. Push it onto the stack if an adjacent point is not blocked and has not been visited
8. Continue until the ending point is found or the stack becomes empty.

9. If the stack becomes empty, no path exists.

d)

1. Start at the starting point.
2. Enqueue into a queue.
3. Remove the front point from the queue while the queue is not empty
4. The search is successful if the point is the ending point
5. Mark it as visited if the point has not been visited
6. Check each valid adjacent point.
7. Then, enqueue it into the queue if an adjacent point is not blocked and has not been visited
8. Continue until the ending point is found or the queue becomes empty.
9. No path exists if the queue becomes empty

e) Trace each search algorithm from S to E on the grid below. Boxes with an X are inaccessible.

Starting point:

S = (0,0)

Ending point:

E = (3,1)

Blocked points:

(3,0), (4,0), (0,1), (2,1), (0,2), (2,2), (3,3), (0,4), (1,4)

Depth-First Search Trace:

(0,0)

(1,0)

(2,0)

(1,1)

(1,2)

(1,3)

(2,3)

(2,4)

(3,4)

(4,4)

(4,3)

(4,2)

(3,2)

(3,1)

DFS finds E at (3,1), so a path exists.

Breadth-First Search Trace:

(0,0)
(1,0)
(2,0)
(1,1)
(1,2)
(1,3)
(2,3)
(2,4)
(3,4)
(4,4)
(4,3)
(4,2)
(3,2)
(4,1)
(3,1)

BFS finds E at (3,1), so a path exists.

4- Hypothesis

If the program uses graph search to check each valid adjacent position in the grid, then it will be able to determine whether a path exists from the starting point to the ending point, because depth-first search and breadth-first search can systematically explore reachable locations while avoiding blocked cells.

5- Step-by-Step Procedure

a) This function checks the valid neighboring positions around point p in the grid g.

If a neighboring point is inside the grid, open, and available to visit, the function inserts that point into the front of the deque s.

This helps the depth-first search choose which point to visit next.

b) This function uses depth-first search to find a path from the start position of the grid to the end position.

It begins at the starting point, stores available points in a deque, and keeps searching through valid adjacent points.

If the end point is found, the function returns the path as a string. If no path exists, it returns an empty string.

6. Implementation Notes

The program was implemented using depth-first search. Deque object was used like a stack by inserting and removing points from the front.

Each search step stored both the current point and the path string used to reach that point.

One important design decision was to create a Step structure that stores a Point and a string path.

This was necessary because storing only Point objects would allow the program to visit locations,

but it would not remember the exact path taken to reach each location.

A bug was encountered where the program originally returned an empty path for every test case.

The reason was that the start point was not treated as an available grid location by the available() method.

I fixed it by allowing the search to process the start point even if available() returned false for it.

7. Code Repository

sample.cpp:

This source file contains the graph search program. It defines the Step structure, the adjacents() function, the pathfinder() function, and the main function with the three required test cases.

Point.h:

This header file defines the Point class. It represents a position in the grid and provides methods such as left(), right(), up(), down(), row(), column(), and set().

Grid.h:

This header file defines the Grid class. It stores the two-dimensional grid, start point, end point, blocked cells, and methods such as load(), available(), mark(), path(), start(), and end().

Deque.h:

This header file defines the Deque class. It is used by the pathfinder function to store points and steps during the depth-first search.

Node.h:

This header file defines the Node class used internally by the Deque class.

Object.h:

This header file defines the base Object class used by other project classes.

8. Output

The program output was stored in separate text files for each test case.

output01.txt:

Test Case 1 - Straight Path

Path: rrrrd

Valid Path: 1

This means the program found a valid path from the start point to the end point.

output02.txt:

Test Case 2 - Verification

Path: rddrrrrulu

Valid Path: 1

This means the program found a valid path from the start point to the end point in the larger 5 by 5 grid.

output03.txt:

Test Case 3 - No Path

Path:

Valid Path: 0

This means the program did not find a valid path, which is correct because the grid blocks access to the end point.

9. Data Analysis

The results show that the depth-first search algorithm correctly handled both reachable and unreachable grid situations.

In Test Case 1, the grid had a simple straight path, and the program returned rrrrd. The path method returned 1, confirming that the path was valid.

In Test Case 2, the grid was larger and contained more blocked cells. The program returned rddrrrrulu, and the path method again returned 1.

This shows that the algorithm can search around obstacles and still locate the end point when a route exists.

In Test Case 3, the program returned an empty path, and the path method returned 0. This was the expected result because no valid path existed.

The main pattern is that valid grids with reachable end points return a path and a validation value of 1, while unreachable grids return an empty path and a validation value of 0.

10. Conclusion

The program successfully solves the original problem of finding whether a path exists from one point in a two-dimensional grid to another point.

By using depth-first search, the program checks valid adjacent positions, avoids blocked cells, and returns a path when one exists. The test cases showed that the program works for a straight path, a more complex grid, and a no-path situation.